

abr. 20, 2026

Integrações Síncronas - Banco de Dados

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider
Gabriel Severo Gabriel kohlrausch Ruan Rodrigues
aline.bianchini@t-systems.com ana.santos@t-systems.com
analista.kuehn@outlook.com anderson.hilario@t-systems.com
anderson.perim@t-systems.com andersongabriel_maria@hotmail.com
anselmogabriel421@gmail.com bmf.pedro@gmail.com
bruna.nery@t-systems.com bruno.destro@t-systems.com
caique.rodrigues@t-systems.com d.silva@t-systems.com
danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br
douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com
eduardo.kubota@t-systems.com eudy.amparo@t-systems.com
fabio.marinelli@t-systems.com farley.t.i@hotmail.com
felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com
hewerton.souza@t-systems.com i.parmigiani@t-systems.com
igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com
jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com
l.regioli@outlook.com leandro.piqueira@t-systems.com
lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com
marcelohcordeiro@gmail.com marcio@almob.com.br
marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com
nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com
regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com
renan-silva@t-systems.com renan.baptista.cabral@gmail.com
rodrigo_7sch@hotmail.com tatidornel@gmail.com
thiago.fernandes@t-systems.com victor.prospero@t-systems.com
victor.santos@t-systems.com victor.souza@t-systems.com
vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com
wesley.boccaletto@t-systems.com wsilva@t-systems.com
fabiano.felgas@gmail.com

Anexos  Integrações Síncronas - Banco de Dados

Resumo

Meeting covered synchronous integration strategies with Entity Framework and Dapper for performance and architectural integrity.

Connection Pool e Performance

O uso adequado do Connection Pool evita a exaustão de recursos. A responsabilidade do desenvolvedor é garantir o descarte correto das conexões via blocos using.

Entity Framework e Dapper

Utilize o Entity Framework para comandos transacionais e o Dapper para consultas de leitura performáticas. Decisão de arquitetura prioriza a eficiência e consistência dos dados.

Boas Práticas de Mapeamento

Centralize configurações na Fluent API para evitar poluição do domínio. A escolha estratégica entre rastreamento e consultas especializadas otimiza o uso de memória e processamento.

Próximas etapas

- ☐ [wallace silva] Confirmar Vazamento: Avaliar aplicação para confirmar ausência de vazamento de conexões.
- ☐ [O grupo] Implementar DynamicParameters: Modificar chamadas Dapper que usam objeto anônimo com filtro string. Utilizar DynamicParameters para tipagem correta.
- ☐ [Natan Barros] Pesquisar EF: Pesquisar sobre concorrência otimista no Entity Framework. Encontrar material oficial da Microsoft sobre o tópico.

Detalhes

- **Introdução às Integrações Síncronas e Connection Pool:** Francisco Schneider iniciou o módulo de integrações síncronas com foco na interação com bancos de dados, enfatizando que o objetivo não é ensinar tecnologias básicas como o Dapper, mas sim discutir problemas recorrentes, "dores" encontradas em clientes e boas práticas ([00:00:00](#)). O ponto de partida foi o Connection Pool, que é fundamental para qualquer framework de conexão em C\# ou .NET. O esperado é que as aplicações abram e fechem muitas conexões idênticas rapidamente, referindo-se à mesma **connection string**, e o **pool** gerencia isso ([00:01:29](#)).
- **Gerenciamento do Connection Pool e Boas Práticas:** O Connection Pool é projetado para mitigar o custo de criação de novas conexões, controlando o tempo de vida e o reuso de conexões, sendo totalmente transparente para a pessoa desenvolvedora. A capacidade de gerenciamento de conexões é nativa do C\# e tem sido aprimorada com as atualizações de versão ([00:02:44](#)). O padrão de fábrica do **pool** é de cem conexões, o que, para a maioria dos cenários, deveria ser suficiente ([00:04:10](#)).
- **Sinais de Alerta no Gerenciamento de Conexões:** Especificar um número máximo de conexões superior a cem pode indicar um problema de gestão, onde as conexões não estão sendo devolvidas ao **pool** corretamente ([00:04:10](#)). Francisco Schneider destacou a recomendação da Microsoft: *"open late and close early"* (abra o mais tarde possível e feche o mais cedo possível) ([00:05:36](#)). Abordagens como criar uma conexão em um **singleton** ou no construtor de uma classe (repositório) são bem-intencionadas, visando o reuso, mas são desnecessárias e podem ser problemáticas, pois o .NET já faz o gerenciamento de forma eficaz ([00:07:08](#)).
- **Obrigação da Pessoa Desenvolvedora: O Uso de Dispose:** A única obrigação da pessoa desenvolvedora para garantir a eficácia do Connection Pool é garantir o fechamento da conexão, o que é melhor feito através do uso de ``using`` ou ``a using`` para objetos que implementam a interface ``IDisposable``. O uso de ``Connection.Close()`` não é estritamente necessário em todos os casos, pois o ``using`` destrói o objeto ([00:08:43](#)). A única exceção notável para a abertura manual de conexão (``*Connection.Open()*``) é ao trabalhar manualmente com transações de banco de dados, fora do Entity Framework (EF) ([00:10:06](#)).
- **Ciclo de Vida da Conexão e do Objeto:** Felipe Neves perguntou sobre o ciclo de vida da variável de conexão e seu descarte ([00:11:17](#)). Francisco Schneider explicou que dar um ``new`` em um objeto de conexão apenas

solicita uma conexão ao `*pool*`, não necessariamente cria uma conexão física nova no banco de dados ([00:12:24](#)). Ao sair do bloco ``using`` (o momento da execução do ``Dispose``), a conexão física não é necessariamente destruída, mas sim retornada ao `*pool*` para que este avalie seu reuso ou descarte posterior, garantindo a saúde da aplicação ([00:13:46](#)).

- **Pool de Conexões em Diferentes Bancos e Escalabilidade:** Gustavo Ferraz Fontes questionou se o gerenciamento do `*pool*` muda dependendo do banco de dados (exemplo: PostgreSQL) ([00:13:46](#)). Francisco Schneider confirmou que o `*pool*` é agnóstico à tecnologia de banco de dados, pois o controle é feito pelo .NET no lado da aplicação, por processo. Essa arquitetura é um fator de risco ao configurar um `*Max Pool Size*` muito alto, pois cada instância da aplicação (cada `*pod*` em um AKS, por exemplo) terá seu próprio `*pool*` de conexões ([00:15:03](#)). Natan Barros compartilhou uma experiência onde diminuir o número de conexões de um `*worker*` liberou o gargalo de performance causado por lentidão no banco ([00:20:56](#)).
- **Consequências de Vazamento de Conexão e Impacto na Performance:** A consequência direta de não realizar o `*Dispose*` corretamente é a exaustão do `*pool*`, o que geralmente resulta em exceções de `*timeout*` ao tentar obter uma conexão ([00:22:09](#)). Antes que o `*timeout*` ocorra, uma má gestão das conexões pode causar lentidão significativa no sistema. Em casos extremos, a lentidão pode ser incorretamente atribuída a uma `*query*` mal escrita, quando na verdade é causada pela indisponibilidade de conexões no `*pool*` ([00:23:30](#)).
- **Problemas de Performance no Dapper e Conversão Implícita:** Ao utilizar o Dapper sem a tipagem explícita dos parâmetros (`*Dynamic Parameters*`), um filtro simples em uma coluna `*varchar*` (tipo comum) fará com que o Dapper envie o parâmetro como `*N varchar 4000*` por padrão ([00:26:33](#)) ([00:31:42](#)). Essa incompatibilidade de tipos causa uma conversão implícita no SQL, forçando um `*Index Scan*` em vez de uma busca eficiente ([00:30:19](#)). Um plano de execução problemático pode ter um custo elevado (exemplo: 629) e exibir alertas de conversão implícita ([00:29:09](#)).
- **Solução e Boas Práticas com Dapper:** Embora a criação de um índice possa melhorar a performance (reduzindo o custo de 629 para 15), ela não resolve a raiz do problema, que é o alerta de `*cast*` implícito ([00:30:19](#)). A solução recomendada é usar `*Dynamic Parameters*` para especificar explicitamente o tipo e, opcionalmente, o tamanho da variável, garantindo que o Dapper envie o parâmetro com o tipo de dados correto para o banco ([00:32:54](#)). Isso resultou na eliminação do `*Index Scan*` (substituído por `*Index Seek*`), setas finas no plano de execução e um custo da `*query*` performática abaixo de zero (exemplo: 0.0065) ([00:38:12](#)). Francisco Schneider alertou que o Dapper,

apesar de ser performático, pode levar a problemas de performance em *queries* simples se usado de forma incorreta ([00:39:33](#)).

- **Otimização e Geração de Queries pelo EF Core:** Francisco Schneider enfatizou a evolução do EF Core em comparação com o EF do *full framework*, notando uma diferença "colossal" na performance e qualidade das queries geradas. As novas versões do EF Core organizam as *queries* geradas para serem mais legíveis e performáticas, corrigindo problemas das versões antigas que geravam *queries* gigantescas e difíceis de entender ([00:58:55](#)).
- **Abordagem Híbrida para Acesso a Dados (EF e Dapper):** A equipe do orador prefere usar o EF para comandos (POST, PATCH, PUT, DELETE) e o Dapper para *gets*. Essa estratégia é justificada, pois o EF já resolve automaticamente o controle transacional e facilita a persistência de agregados em *Domain-Driven Design* (DDD), enquanto o Dapper é valorizado por seu poder de performance em operações de recuperação de dados simples ([01:00:24](#)).
- **Desmistificando o Desempenho do Entity Framework:** Francisco Schneider abordou o estigma de que o EF não é performático, explicando que essa percepção era mais justificada em versões antigas que geravam *queries* ruins. Atualmente, se houver lentidão com o EF, isso pode indicar que ele está sendo usado em um contexto inadequado ou de maneira incorreta, assim como o Dapper pode causar problemas se usado de forma errada ([01:01:48](#)).
- ****Configuração *Default* e Implicação de Logs Limitados**:** A configuração *default* do EF, embora seja a mais simples, assume implicitamente que a aplicação não usará retentativas e terá logs limitados. logs limitados podem ser inconvenientes em cenários de *troubleshooting* ([01:03:14](#)).
- **Melhorias na Configuração do EF: Retentativas e Rastreamento:** É considerado fundamental configurar retentativas para o EF, sendo o "arroz com feijão" para quem trabalha com a ferramenta ([01:04:42](#)). As retentativas são destinadas a erros transientes (problemas de rede ou indisponibilidade momentânea), e não a erros de sintaxe ou *timeouts* ([01:03:14](#)). O rastreamento padrão usa o *change tracker* e pode ser alterado, mas isso não é recomendado ([01:04:42](#)).
- ****Política de Retentativas e Risco de *Timeout* da API**:** Felipe Neves levantou a preocupação de que a política de *retry* configurada no EF possa entrar em conflito com o *timeout* da API. Francisco Schneider concordou que o *timeout* pode ocorrer, mas remover o *retry* não resolveria o problema subjacente ([01:06:07](#)). Eles poderiam configurar o *max try delay*

do `*retry*` para ficar abaixo do `*timeout*` da API para mitigar esse risco ([01:07:28](#)).

- ****Retentativas e Gerenciamento de Conexões (*Connection Pool*)****: Felipe Neves questionou se a política de `*retry*` poderia comprometer o processamento ou esgotar o `*connection pool*` ([01:07:28](#)). Francisco Schneider afirmou que o EF lida com as conexões de forma correta e que o `*retry*` não deveria causar o esgotamento do `*pool*`, uma vez que as retentativas são cenários de exceção e não devem ser acionadas com frequência ([01:08:45](#)).
- ****Explicação do *Change Tracker* (Rastreamento)****: Marcelo Henrique solicitou uma explicação sobre o `*change tracker*`, já que eles usam a opção ``.AsNoTracking()`` sem entender completamente o motivo ([01:08:45](#)). Francisco Schneider explicou que, por padrão, o EF rastreia objetos em memória para saber o que precisa ser persistido (INSERT, DELETE, UPDATE) no `*save changes*`. O mecanismo de `*change tracker*` armazena dados adicionais em memória para esse controle ([01:11:49](#)).
- ****Uso de `\*.AsNoTracking()\*` e Economia de Recursos****: A regra de ouro para usar ``.AsNoTracking()`` é quando um objeto está sendo recuperado e não haverá alteração de seus dados. O uso de ``.AsNoTracking()`` evita o `*overhead*` de processamento e alocação de memória adicional gerada pelo `*change tracker*` ([01:12:59](#)).
- *****Tracking* vs. *Lazy Loading*****: Felipe Neves pediu uma distinção entre `*tracking*` e `*lazy loading*` ([01:14:24](#)). Francisco Schneider esclareceu que o `*tracking*` se refere ao rastreamento do estado do objeto para alteração. O `*AsNoTracking*` é usado para objetos somente leitura, como ao recuperar uma configuração ou validar dados, onde não há intenção de alterar o objeto naquele `*endpoint*` ([01:15:37](#)). `*Lazy loading*` é sobre materializar o dado no último momento possível, e são conceitos diferentes ([01:16:54](#)).
- **Configuração Padrão de Rastreamento**: Farley Souza questionou se seria viável inverter o comportamento padrão para que todas as `*queries*` fossem `*no tracking*`. Francisco Schneider explicou que isso inverteria o padrão de uso, exigindo ``.AsTracking()`` para os casos de rastreamento, o que não é uma boa prática e não faz sentido, pois o padrão de mercado é usar ``.AsNoTracking()`` apenas onde não se deseja alteração de estado ([01:16:54](#)).
- ****Uso de *Retry* em Bancos de Leitura****: Jonathan Jimenez relatou que a equipe dele utiliza a configuração de `*retry*` ao separar bancos de escrita e leitura. Eles apontam a configuração para o banco de leitura, pois não haverá escrita, fazendo com que todas as consultas sejam realizadas com `*no tracking*` ([01:18:02](#)).

- ****Monitoramento das Retentativas (*Retry*)****: Eduardo Kubota perguntou como saber se e quantas retentativas ocorreram. Francisco Schneider respondeu que, se a retentativa for bem-sucedida, não há certeza se isso é explicitamente logado, mas se falhar, o desenvolvedor terá uma **exception** e saberá que ele tentou e não conseguiu alcançar o banco ([01:19:10](#)).
- ****Eficiência do *Retry* e *Connection Pool*****: Francisco Schneider reafirmou que o **retry** do EF é diferente de uma conexão travada; ele tenta novamente apenas em casos de erros transientes e não em **queries** lentas ou falhas de sintaxe ([01:20:23](#)). O EF gerencia o **connection pool**, sendo um benefício de usar um ORM como o EF, em contraste com o Dapper, que exigiria controle manual de **retries** e conexões ([01:21:36](#)).
- : O **DB Context Pool** é um recurso que habilita o reaproveitamento de instâncias do DB Context, um objeto que é "pesado e caro" para criar e destruir ([01:23:50](#)). O **Pool** não deve ser usado se o DB Context contiver informações personalizadas, como um **Tenant ID**, pois essas propriedades podem vazar para outras requisições ([01:25:11](#)).
- ****Exemplos de Não Uso do *DB Context Pool*****: Gustavo Ferraz Fontes perguntou quais customizações poderiam vazar, e Francisco Schneider citou o **Tenant ID** como um exemplo de propriedade personalizada que, se presente no DB Context, desaconselha o uso do **Pool** ([01:26:57](#)). Propriedades personalizadas podem reter "sujeira" de uma requisição anterior ([01:25:11](#)) ([01:28:04](#)).
- : Jonathan Jimenez usa o ``AsNoTrackingWithIdentityResolution()` para resolver problemas ao buscar grandes listas com dependências na mesma pessoa, pois isso garante que o objeto será consultado apenas uma vez, economizando memória. Esse recurso é uma variação do **no tracking** que garante que, se a mesma instância for materializada mais de uma vez, ela reutilize a mesma referência em memória ([01:30:46](#)).
- ****Benefício e Escopo do *Identity Resolution*****: O principal benefício do **Identity Resolution** é a redução da pressão na memória e um **garbage collection** mais suave, evitando a criação de múltiplas instâncias idênticas em memória, como em um **join** onde uma categoria está em vários produtos ([01:32:03](#)). Caique CR perguntou se faria sentido usar o **Identity Resolution** em um **select** de tabela única. Francisco Schneider explicou que não faz sentido, pois a duplicidade só ocorre em **joins** onde a mesma entidade é materializada várias vezes ([01:34:51](#)).
- ****Implementação Incorreta de Persistência (*Save Changes* no Repositório)****: Francisco Schneider apresentou um repositório onde cada operação (**update**, **delete**) já chamava o **save changes** ([01:38:47](#)).

Gustavo Ferraz Fontes identificou que o problema é o `*save changes*` dentro de cada método, fechando a transação precocemente. Essa abordagem desperdiça o controle transacional robusto que o EF fornece ([01:40:16](#)).

- ****Abordagem Incorreta de `*Update*` no Repositório****: Francisco Schneider também criticou a presença do método `*Update*` no repositório ao usar o EF ([01:40:16](#)). O método `*Update*` é inútil na maioria das vezes, a não ser que uma entidade tenha sido recuperada com `*no tracking*` e precise ser rastreada manualmente ([01:41:37](#)). O `*Update*` no EF apenas avisa o `*change tracker*` em memória, sendo que atualizações devem ser feitas recuperando a entidade com `*tracking*`, alterando as propriedades e chamando `*save changes*` ([01:42:51](#)).
- ****Problemas de Múltiplos `*Commits*`****: A persistência feita com múltiplos `*commits*` acarreta baixa consistência, dificuldade de `*rollback*`, degradação de performance (devido ao vaivém com o banco) e complexidade. A solução para isso é o padrão `*Unit of Work*` ([01:44:19](#)).
- ****Padrão `*Unit of Work*` (`*By the Book*`)****: O padrão `*Unit of Work*` (UoW) `*by the book*` envolve um `*envelope*` que encapsula repositórios e o DB Context, garantindo que o cliente passe sempre pelo UoW para acessar o banco ([01:44:19](#)). O UoW expõe um método de `*commit*` (ou `*save changes*`) para persistir todas as operações de uma só vez ([01:45:57](#)).
- ****Abordagem Alternativa: DB Context como `*Unit of Work*`****: Gustavo Ferraz Fontes e Francisco Schneider concordaram que a abordagem `*by the book*` não é elegante ([01:47:38](#)). Eles defendem que, como o próprio DB Context do EF já é um `*Unit of Work*` por natureza (entregando atomicidade transacional), faz sentido que ele implemente a interface `*Unit of Work*` ([01:49:02](#)).
- **Benefícios de Estender o DB Context para UoW**: Estender o DB Context para implementar o UoW permite adicionar lógica de infraestrutura antes do `*save changes*`, como a publicação automática de eventos de domínio ou o gerenciamento de campos como `*data de criação*` ou `*data de alteração*`, sem poluir o domínio ([01:50:28](#)).
- ****Injeção do `*Unit of Work*`****: Vanderlei Henrique levantou a preocupação de que expor o UoW através de um repositório (ex: ``orderRepository.unitofwork.saveEntities``) pode ser menos natural e causar confusão ([01:51:48](#)). Francisco Schneider e Gustavo Ferraz Fontes concordaram, preferindo injetar a interface `*Unit of Work*` diretamente nos `*handlers*` para maior legibilidade e para enfatizar que o `*save changes*` é atômico para todo o contexto ([01:53:13](#)).

- **Controle Transacional na Camada de Aplicação (Má Prática):** Marcelo Henrique mencionou ter visto clientes onde o controle transacional e as *queries* eram feitas na camada de lógica de negócios, fora dos repositórios ([01:55:26](#)). Francisco Schneider classificou isso como uma abordagem ruim, pois gera alto acoplamento com o banco de dados e dificulta a escrita de testes e o reuso de código, desrespeitando a encapsulação na camada de infraestrutura ([01:57:02](#)).
- **Discussão sobre Código Não Sustentável e Práticas Legadas:** Francisco Schneider e Marcelo Henrique discutiram a natureza insustentável de certos códigos, que frequentemente são traduções de linguagens antigas da empresa, como a conversão de Visual Fox Pro para C\#ARP, replicando heranças procedurais em vez de aproveitar os benefícios da linguagem mais recente ([01:58:27](#)). Foi observado que essa prática é um erro básico de fundamentos e viola o princípio SOLID, pois a camada de aplicação deve atuar como um orquestrador e não se preocupar com a abertura de conexão com o banco de dados. Marcelo Henrique buscou esclarecimentos por curiosidade, confirmando que a prática não é recomendada, mesmo com o Entity Framework (EF), que abstrai essas preocupações ([01:59:40](#)).
- **Utilização de Interceptors do Entity Framework:** Gustavo Ferraz Fontes perguntou a Francisco Schneider sobre a prática de substituir funcionalidades do `SaveChanges` pelos Interceptors do EF, mencionando o uso para resolver casos de *outbox* e *log* de auditoria ([02:00:38](#)). Francisco Schneider expressou que gostam da ideia dos Interceptors, mas aconselham cautela para não sobrecarregar eles com muitas funcionalidades, o que poderia prejudicar o *troubleshooting*, uma vez que o código fica implícito ([02:01:51](#)). Interceptors são úteis para abstrair complexidade e podem ser usados para operações como definir datas de cadastro e atualização ([02:03:03](#)).
- ****Tratamento de *Race Condition* com Entity Framework**:** Natan Barros levantou uma questão sobre o tratamento de *race condition* (condição de corrida) com o Entity Framework, sugerindo a abordagem de usar o método `ExecuteUpdate` para verificar a quantidade de linhas alteradas ([02:03:03](#)). Francisco Schneider explicou que o EF oferece uma *feature* pronta para isso, conhecida como concorrência otimista, que envolve a criação de uma coluna de versão no banco de dados (como `row version` no SQL) ([02:04:21](#)). O EF compara essa versão automaticamente durante o *commit* para garantir que apenas uma transação seja bem-sucedida, resolvendo a concorrência de forma mais eficiente do que uma abordagem manual ([02:05:33](#)).
- **Concorrência Otimista em Agregados:** Natan Barros questionou a eficácia da concorrência otimista em um fluxo de negócio complexo que envolve um

agregado com múltiplas entidades relacionadas dentro de uma mesma transação ([02:05:33](#)). Francisco Schneider explicou que o controle de concorrência deve ser mantido no agregado pai (o principal), não nas entidades filhas, e que a abordagem do EF resolve esse cenário quando aplicada ao pai ([02:06:55](#)). A abordagem manual exige que o desenvolvedor recupere um **token** (como o estado inicial do agregado) e o use como condicional no momento do **update** ([02:08:14](#)).

- **Mapeamento de Propriedades e Data Annotations:** Francisco Schneider introduziu o tópico de mapeamento de propriedades, observando que é possível persistir uma classe simples no banco sem configurações adicionais, mas que o EF assume padrões ([02:08:14](#)). Eles argumentaram que usar *`Data Annotations`* na entidade de domínio (para especificar PK, nome de tabela, etc.) é uma má prática, pois "suja" o domínio com informações de banco de dados e cria propriedades de navegação que não fazem sentido no contexto do domínio, como um *`Post`* ter uma referência a um *`Blog`*. A solução para evitar essa sujeira é o uso da Fluent API ([02:09:36](#)).
- **Mapeamento Moderno com Fluent API e Conversores Personalizados:** Francisco Schneider demonstrou o mapeamento moderno com a Fluent API, centralizando as configurações em classes de mapeamento separadas, em vez de agrupar tudo no *`OnModelCreating`*. Eles enfatizaram a importância de especificar tipo e tamanho (para **strings** e **ints**) para ajudar o EF a gerar **queries** mais performáticas ([02:12:26](#)). Conversores nativos do EF podem ser usados para traduzir **enums** para **strings** no banco, e conversores personalizados foram utilizados para criptografar dados bancários, garantindo que o domínio e a camada de aplicação não precisem se preocupar com a criptografia na persistência ([02:13:39](#)).
- ****Mapeamento de Relacionamentos Um-para-N e *Structs***:** O mapeamento de relacionamentos Um-para-N (como Assinatura e Beneficiários) com a Fluent API foi detalhado, mostrando como especificar a Chave Estrangeira (FK) e configurar o comportamento de exclusão em cascata, resolvendo a relação sem sujar a classe de domínio filha ([02:16:29](#)). Além disso, Francisco Schneider mencionou que o EF mais recente permite o uso de **structs** como **value objects**, o que era uma limitação anterior, e que essa **feature** elimina desculpas para não usá-los ([02:18:11](#)).
- **Erros Comuns em Consultas do Entity Framework (Massive Includes):** Francisco Schneider discutiu o problema de uma quantidade massiva de *`Includes`* nas consultas, que é um erro comum, especialmente em código legado, resultando em **queries** lentas e recuperação excessiva de dados ([02:19:27](#)). Essa prática leva a problemas de performance, disponibilidade e **timeout**, pois o **change tracker** precisa rastrear uma quantidade grande de

objetos, e o ideal é ter métodos especialistas que retornem apenas o que é necessário, evitando o reuso de métodos genéricos que recuperam dados em excesso ([02:20:38](#)).

- **Erros Comuns em Consultas do Entity Framework (Select Asterisk):** Outro erro comum é o ``SELECT *`` implícito, que ocorre quando o desenvolvedor recupera todos os dados de um objeto (*e.g.*, ``Posts.Where().ToList()``) e usa apenas uma pequena fração deles. Essa conveniência de não criar materializações diferentes resulta em grande quantidade de dados sendo trazida para a memória ([02:24:19](#)). A solução é criar métodos especialistas que retornem um `*DTO*` (ou objeto similar) com apenas os dados necessários, melhorando a performance, embora exija mais trabalho ([02:25:47](#)).
- **Uso de Tags em Consultas para Troubleshooting:** Francisco Schneider recomendou o uso da `*Tag With*` no EF para correlacionar `*queries*` complexas com o código-fonte, o que é muito útil para `*troubleshooting*` ([02:29:06](#)). Eles preferem usar um método que gera logs com o nome do arquivo e o número da linha da `*query*`, pois isso facilita a localização do problema, sem ônus de performance ([02:30:01](#)).
- **Erros Comuns em Consultas do Entity Framework (Contagem Ineficiente):** Um erro de performance comum é materializar a lista completa de resultados para realizar uma contagem (como ``ToList().Count``), o que é ineficiente. A solução é usar métodos performáticos do EF, como ``AnyAsync``, que o DBA considera altamente performático por gerar a `*query*` SQL ``EXIST`` ([02:31:24](#)). A premissa de que o EF é lento geralmente se deve à má escrita do código, e não à culpa do `*framework*` ([02:32:31](#)).
- **Comparação de Consultas em LINQ vs. Raw SQL e Recomendações de Mapeamento:** Farley Souza questionou o uso de consultas complexas em LINQ, ao que Francisco Schneider respondeu que a prática é aceitável se o time tiver a `*skill*` para isso, mas que a preferência pessoal é contra, pois pode se tornar confuso para desenvolvedores menos familiarizados ([02:33:44](#)). Sobre o mapeamento relacional, Francisco Schneider defendeu a centralização de todas as configurações via Fluent API nas classes de mapeamento, o que garante organização e facilita a manutenção, pois o desenvolvedor sabe exatamente onde procurar em caso de problemas ([02:36:31](#)).
- **Relação entre Mapeamento e Select:** Francisco Schneider corrigiu a percepção de que um bom mapeamento via Fluent API implica que o Entity Framework sempre trará o agregado completo no `*select*`, afirmando que isso não está correto. O mapeamento é crucial para a persistência correta e

atualização do agregado (inserção, atualização, deleção), mas o desenvolvedor deve decidir no `*select*` o que materializar (por exemplo, usando `*DTOs*`) ([02:37:43](#)). Jonathan Jimenez complementou que isso tem a ver com a configuração de `*lazy loading*` no EF ([02:40:15](#)).

- **Decisão entre Entity Framework e Dapper (CQRS Approach):** Eduardo Kubota questionou como decidir entre usar EF ou Dapper no design de uma aplicação. Francisco Schneider recomendou usar o EF para a camada de comando (alterar estado, persistir dados, operações transacionais) e o Dapper para a camada de `*query*` (operações ``GET`` e leituras simples), uma abordagem básica que resolve bem ([02:41:30](#)). Apesar de ser possível escrever `*queries*` performáticas com o EF, o Dapper é inerentemente mais performático para `*queries*` simples devido à natureza pesada do ``DbContext``. Eles observaram que a maioria dos times têm mais dificuldade em usar o EF corretamente do que o Dapper ([02:43:09](#)).
- **Considerações Finais sobre EF e Dapper:** Francisco Schneider concluiu que, em termos transacionais e no trabalho com agregados, o EF é mais fácil e oferece mais garantias. No entanto, para consultas extremamente performáticas ou `*selects*` simples, o Dapper é uma opção mais rápida, embora não seja uma regra rígida ([02:44:30](#)). Em cenários de alta frequência de criação de ``DbContext``, o uso do `*pool*` de ``DbContext`` com o EF é altamente recomendado para aprimorar a performance ([02:45:56](#)).

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações](#)

Como está a qualidade de **destas observações**? [Responda a uma breve pesquisa](#) para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.